

```
type name [ elements] ;
```

Here, `type` refers to the data type such as `int` and `float`. `name` refers to a valid identifier, while the `elements` field indicates the numbers of elements in the array. You can declare the above case as:

```
int Jim[ 6] ;
```

While declaring a regular array, if you don't specify the initial values of the elements, the elements will not be initialised to any value by default. The elements of global and static arrays are initialised automatically with their default values, i.e. 0. When an array is declared, you can assign initial values to each element by enclosing the values within braces `{ }`. For example,

```
int Jim[ 6] = { 12,34,56,78,12,32} ;
```

The number of values inside the braces should never be larger than the number of elements declared for the array in square `[ ]` brackets. C++ also allows the square brackets to remain empty of initialisation values for certain situations. For this situation, the compiler assumes the size of an array that matches the number of values included within the braces. This is shown below:

```
int Jim[ ] = { 212,234,456,781,10,312} ;
```

At any point in a program, where an array is visible, the value of any individual elements can be accessed, as if it's a normal variable. The syntax for this is:

```
name [ index]
```

For example, consider the case in which an array named `Jim` has six elements and each of those elements is of type `int`. If we want to store the value 60 in the third element of the array `Jim`, we can write the following statement: